SARRTHI: A Proactive Compliance and Payment-Monitoring Platform for MSME Suppliers to Government Transport Bodies

Design, Implementation and Evaluation of a Demo-Mode Prototype

Research paper prepared for academic project submission, October 2026. Author(s): [your name(s), programme and institution]

Abstract

Micro, Small and Medium Enterprises (MSMEs) that supply components, fabrications and services to Indian Railways and comparable government transport bodies are protected on paper by the MSMED Act, 2006, which sets a 45-day payment window and a compound-interest remedy, and by a 20% procurement quota. In practice, vendors learn that a payment is late, or that a quota or specification rule has been breached, only after working capital has already been locked up. The government's dispute mechanism, the MSME Online Dispute Resolution (ODR) Portal, activates only after a default has occurred. This paper presents SARRTHI, a proactive compliance and payment-monitoring platform positioned upstream of that mechanism. We first derive the problem from public evidence and a gap analysis of existing government tools, then formalise it as a Software Requirements Specification (SRS) with nine functional modules and a four-phase delivery plan. We then describe the design and implementation of a working prototype: a React and TypeScript single-page application backed by a Motoko canister on the Internet Computer, joined by a data-source abstraction that lets the same screens run against browser-persisted demo data or a configurable REST API. The prototype implements a configurable payment-window monitor, a transparent interest estimator that states its assumptions, a rule-based pre-bid risk score, buyer delay reporting, a working-capital gap estimate, an evidence-oriented document library, and CSV and print reporting. We document the algorithms with worked numerical examples, relate every prototype feature to the SRS, report the structure of an automated test suite of 77 test cases, and give an honest account of limitations, including the gap between the SRS's Java Spring Boot mandate and the implemented stack, and several defects found by code inspection. The paper closes with a research agenda covering statutory compound interest, real procurement data, collective intelligence and predictive risk models.

Keywords: MSME, delayed payments, MSMED Act, compliance monitoring, procurement, invoice tracking, risk scoring, React, Motoko, Internet Computer, requirements engineering

1. Introduction

1.1 Background

Micro, Small and Medium Enterprises form the backbone of Indian manufacturing and of the supply chains that feed public infrastructure. India has roughly 6.4 crore registered MSMEs, which contribute over 4.6% of Gross Value Added according to the figures compiled in this project's background research. Since 1 April 2025 the classification thresholds under the MSMED Act were revised: a micro enterprise has plant and machinery investment up to ₹2.5 crore and turnover up to ₹10 crore, a small enterprise up to ₹25 crore and ₹100 crore, and a medium enterprise up to ₹125 crore and ₹500 crore [2].

Government departments and public sector undertakings, including Indian Railways, are among the largest buyers from this sector. Railways sources rolling-stock parts, castings, forgings and fabricated components from thousands of vendors through its e-procurement system and through the Government e-Marketplace (GeM), where tens of thousands of MSME sellers were reported active [10]. Two statutory obligations govern this relationship: a payment must be made within 45 days of the date of acceptance of goods or services (Section 15 of the MSMED Act), with compound interest at three times the RBI-notified bank rate payable on delay (Section 16) [1]; and a minimum of 20% of annual purchases is reserved for micro and small enterprises.

1.2 Motivation

The law exists, but compliance is unevenly observed and, more importantly, unevenly *visible*. National estimates place delayed MSME payments at about ₹7.34 lakh crore for FY24 [5], and government sources have named Railways among the sectors owing the most to MSMEs. A late payment is also not an isolated loss. It triggers a chain of secondary harms: interest entitlements that vendors rarely calculate or claim, bank accounts that drift toward Non-Performing Asset classification after 90 days of default because a buyer paid late, and tax effects under Section 43B(h) of the Income Tax Act that give buyers a reason to pay on time but only if the delay is documented [9].

The common thread is an information failure. The facts needed to prevent or contest a loss (the acceptance date of an invoice, the days remaining, the interest accrued, the buyer's history of delay) usually exist somewhere in the vendor's own records, yet no tool assembles them into a live, vendor-facing picture before a dispute becomes necessary.

1.3 Problem Statement

MSME vendors supplying government transport bodies have no independent, self-serve system that lets them:

1. track, in real time, whether each invoice is approaching or has breached the statutory payment window;
2. see the interest liability accruing on late invoices, with its assumptions stated;
3. judge a buyer's payment reliability and their own documentation readiness *before* bidding;
4. keep clean, dispute-ready evidence for every order and invoice; and
5. forecast the working-capital gap created by unpaid invoices.

Existing remedies activate after default. This paper addresses the preventive gap that lies between award of a contract and the point at which a formal dispute is needed.

1.4 Objectives

The project had five objectives:

- **O1.** Establish the problem and the gap in existing mechanisms from public evidence.
- **O2.** Specify a complete, phased set of requirements for a platform that closes the gap.
- **O3.** Implement a working prototype of the Phase 1 scope and the first elements of Phase 2.
- **O4.** Make every computed figure transparent, labelled as an estimate, and configurable where the law is ambiguous.
- **O5.** Evaluate the prototype honestly against its specification, and record its limitations.

1.5 Contributions

The paper makes the following contributions.

1. A **gap analysis** of the MSME ODR Portal against a proactive-monitoring feature set, showing that the government system and the proposed platform are complements rather than competitors (Section 2).
2. A **structured SRS** of nine modules, with a tagging scheme that separates original ideas from necessary additions and advanced upgrades, and a four-phase delivery matrix (Section 3).
3. A **layered architecture** for the prototype, including a *data-source abstraction* that lets one UI run against demo data or a live API without code changes in the screens (Section 4).
4. **Transparent algorithms** for payment-window monitoring, interest estimation, buyer delay aggregation, working-capital estimation and pre-bid risk scoring, each with stated assumptions and worked examples (Section 5).
5. A **traceability analysis** of prototype against SRS, and a candid list of limitations and defects (Sections 3.7, 8 and 9).

1.6 Scope and Disclaimer

SARRTHI is an academic project. It is not an official government portal, uses no government logos, carries no claim of affiliation, and shows the disclaimer "Academic Project — Not an Official Government Portal" persistently in the application footer. It performs no arbitration, mediation, legal advice or any function reserved for the MSME ODR Portal. All interest and risk figures are estimates and monitoring indicators, not legal determinations. The prototype runs on realistic but fictional sample data.

1.7 Organisation of the Paper

Section 2 reviews the legal and institutional background and related mechanisms. Section 3 presents the requirements. Section 4 describes the architecture and data model. Section 5 explains the core algorithms. Section 6 walks through the implemented modules. Section 7 describes testing and evaluation. Section 8 discusses findings, Section 9 limitations and threats to validity, Section 10 future work, and Section 11 concludes. Appendices give a data dictionary, the API surface, the sample data and a proposed mapping to the Java Spring Boot reference architecture.

---

2. Background and Related Work

2.1 The Statutory Framework

The MSMED Act, 2006 gives suppliers three protections relevant here [1].

Payment window (Section 15). A buyer must pay within the period agreed in writing, which may not exceed 45 days from the day of acceptance or deemed acceptance. Where there is no written agreement, the default period is 15 days. This asymmetry is practically important: a vendor who accepts a purchase order without a written payment term silently loses protection. The SRS therefore includes a *contract-clause checker* that flags missing written payment terms.

Interest on delay (Section 16). Delayed payment attracts compound interest with monthly rests at three times the bank rate notified by the Reserve Bank of India. Vendors seldom compute this figure themselves, so the entitlement goes unclaimed.

Dispute resolution (Sections 18 to 21). Disputes go to the Micro and Small Enterprise Facilitation Council (MSEFC) for conciliation and arbitration. An appeal against an award requires the buyer to deposit 75% of the awarded sum. Researchers and trade bodies have noted that this safeguard can also discourage large buyers from contracting with registered MSMEs at all.

Two further provisions complete the picture. Section 43B(h) of the Income Tax Act, inserted by the Finance Act, 2023, denies a buyer a tax deduction for a delayed MSME payment until the year it is actually paid [9]. And banking norms classify an account as a Non-Performing Asset after 90 days of default, so a government buyer's delay can damage the supplier's own credit standing.

2.2 Existing Government Mechanisms

The **MSME Samadhaan** portal, launched on 30 October 2017, let MSMEs file delayed-payment complaints with the state MSEFC and track their status. It stopped accepting new filings on 15 October 2025 and now serves as a status archive [4]. Its historical statistics are sobering: a backlog of roughly ₹21,108 crore across more than 90,000 applications, and a reported 14,739 of 18,203 applications in one sample receiving no response at all from the buying body. These figures come from different snapshots and are not additive, but together they show persistence rather than a single reconciled total.

The **MSME ODR Portal**, launched on 27 June 2025 and mandatory for new delayed-payment cases from 15 October 2025, introduced a two-stage process [3]. Stage 1 is a voluntary, confidential negotiation or mediation with both parties' consent. Stage 2 is the statutory MSEFC conciliation and arbitration process, entered automatically if Stage 1 fails. Awards carry the force of a court decree and are enforceable through the District Collector. The portal includes an interest calculator, case tracking, an aggregate statistics dashboard and limited financial assistance. Only micro and small enterprises with valid Udyam registration can use it; medium enterprises are excluded from the whole mechanism.

2.3 Financing Instruments

The Trade Receivables Discounting System (TReDS) allows a supplier to sell an accepted invoice to a financier at a discount and be paid early. The infrastructure exists, but vendors rarely invoke it proactively. The SRS therefore treats TReDS as a *referral target*: SARRTHI would trigger the referral once an invoice sits unpaid past a threshold, rather than reimplementing discounting.

2.4 Feature Compatibility and Gap Analysis

Table 1 compares the ODR Portal with the proposed platform.

Dimension	MSME ODR Portal	SARRTHI
Point of activation	After a default has occurred	Before a default occurs
Core function	Negotiation, conciliation, arbitration	Monitoring, alerting, risk visibility
45-day clock visibility	Not tracked before filing	Tracked live from acceptance
Interest calculation	At time of filing	Continuous, visible before filing
20% quota monitoring	Not covered	Estimated per buyer or zone
Specification-change alerts	Not covered	Proposed (crowd-sourced, later phase)
Buyer payment-risk profile	National aggregates only	Per-buyer view for bidding decisions
Eligible users	Micro and small only	Micro, small and medium

Dimension	MSME ODR Portal	SARRTHI
Legal force of outcome	Binding award	None; feeds into ODR

Table 1. Compatibility matrix of the government portal and the proposed system.

Six gaps remain even after the 2025 upgrade: (i) no proactive tracking of a vendor's own invoices against the window; (ii) no independent evidence base for quota complaints; (iii) no early warning of retroactive specification changes; (iv) no comparative view of buyer reliability before bidding; (v) no institutional recourse at all for medium enterprises; and (vi) no integration between a vendor's own records and the dispute system, so a vendor must notice a problem unaided before any government tool becomes relevant.

2.5 Positioning

SARRTHI is deliberately *upstream and complementary*. It monitors, calculates and warns. It does not resolve disputes. Where a dispute becomes unavoidable, the intended path is an "ODR pre-fill" that assembles already-tracked data for filing, reducing effort rather than competing with the statutory mechanism. This boundary is also a design constraint: the application must never be presented as giving legal advice.

2.6 Research Questions

The work is organised around four questions.

- **RQ1.** Can a vendor-supplied-data-only system deliver useful proactive monitoring without any government data integration?
- **RQ2.** How can a legally ambiguous quantity, the start of the 45-day clock, be represented so the user can see and change the assumption?
- **RQ3.** Can transparent, rule-based scoring give a vendor an actionable pre-bid signal without a trained model or external data?
- **RQ4.** Which architectural choices let the same interface serve both an offline demonstration and a future production backend?

---

3. Requirements Analysis

3.1 Method

Requirements were consolidated from the project's research outputs (a problem statement and feasibility report, a losses and problems report, and a feature roadmap) into a single SRS [11] structured after the IEEE and ISO/IEC/IEEE 29148 guidance [18]. Each feature carries one of three origin tags: *Original* (part of the initial idea), *Added-Necessary* (required for the product to work as a complete, deployable system) and *Advanced* (an upgrade of an original feature). The SRS also fixes a backend technology mandate of Java 17 with Spring Boot, which Section 4 returns to.

3.2 User Classes

User class	Description
MSME Owner	Primary account holder with full access, billing and data-sharing consent
MSME Staff	Accountant or office staff; invoice entry and document upload only
Platform Administrator	Internal team; verification, moderation, support
Trade Body (future)	Read-only access to aggregated, anonymised non-compliance data
Lending Partner (future)	Consent-gated, API-based access to a vendor's compliance history

Table 2. User classes defined in the SRS.

3.3 Functional Modules

The SRS organises functionality into nine modules.

Module	Purpose	Representative features
1. User, Identity and Access	Verified, role-restricted access	Udyam verification, RBAC, JWT authentication, consent management

Module	Purpose	Representative features
2. Invoice and Payment Monitoring (core)	The preventive core	Payment-clock tracker, interest calculator, contract-clause checker, document-completeness check, working-capital forecaster
3. Compliance Intelligence	Buyer and quota visibility	Quota dashboard, specification-change feed, buyer risk profile, pre-bid risk score, buyer rating loop
4. Communication and Escalation	Act at the right time	WhatsApp alerts at day 30 and 40, auto-drafted escalation ladder, ODR pre-fill, vernacular rights chatbot, multi-language UI
5. Document and Data Capture	Evidence and ingestion	Document vault, OCR upload, Tally integration, GST e-invoice integration
6. Financial Support	Cushion the damage	TReDS referral, exportable PDF and Excel compliance reports
7. Collective Intelligence	Network effects	Crowd-sourced quota tracker, spec-change alert network, collective escalation bundling, trade-body handoff
8. Reporting, Admin and Accessibility	Operate and trust	Admin dashboard, audit-trail export, offline-first mode, encryption and audit logging
9. Monetisation	Sustainability	Freemium tiering, lender-facing API

Table 3. Functional modules of the SRS.

The SRS adds a useful implementation note: the single-vendor quota dashboard and specification-change feed are the *small-network* versions of two Module 7 features, so one underlying service should be built for each pair, with crowd-sourced data simply improving accuracy as more vendors join.

3.4 Non-Functional Requirements

Category	Requirement
Performance	API response under 2 seconds for 95% of requests under normal load
Security	AES-256 at rest, TLS 1.2 or later in transit, role-based access enforced at the API layer
Scalability	Modular monolith whose modules can be extracted into services without redesign
Availability	99.5% uptime target for tracking and alerting
Usability	Core workflows within three user actions from login
Compliance	Digital Personal Data Protection Act, 2023, including deletion and consent withdrawal [22]
Maintainability	Single-language backend and linted, consistent code style
Portability	Containerised deployment across cloud providers

Table 4. Non-functional requirements.

3.5 Phasing

The SRS's delivery plan is deliberately conservative about uncertain dependencies.

- **Phase 1, MVP:** Module 1 (full), Module 2 (full), OCR upload and the document vault, the admin dashboard and security basics. It needs only vendor-supplied data.
- **Phase 2, core expansion:** Module 3, Module 4, the TReDS referral, and Tally and GST integration.
- **Phase 3, growth engine:** Module 7 and freemium tiering, which require a critical mass of vendors.
- **Phase 4, advanced:** AI default prediction, predictive cash-flow analytics, a voice assistant, blockchain document proofs, direct government data integration and native apps.

3.6 Constraints and Assumptions

Three constraints shape everything that follows. First, no confirmed public API exists for the Indian Railways e-procurement system, and even internal vigilance officials have needed a specific administrative order to view finalised tender records. The first version must therefore run entirely on **vendor-supplied data**. Second, the system must comply with the DPDP Act and must not claim any legal or arbitral function. Third, the SRS assumes vendors will enter or connect their own invoice and purchase-order data; the platform cannot function without that minimum input.

3.7 Traceability: SRS to Prototype

Table 5 relates the SRS to what the prototype actually implements. "Implemented" means working code and screens; "Partial" means a simplified or data-model-only version; "Not implemented" means outside the prototype's scope.

SRS feature	Status	Where in the prototype
Payment-clock tracker (45-day window)	Implemented	Configurable monitor in the interest utilities and the invoice screens
Interest-liability calculator	Partial	Simple-interest estimate with stated assumptions; statutory compound interest is future work
Working-capital gap forecaster	Partial	Entered-expense estimate on the reports screen
Document vault and evidence checklist	Partial	Metadata records by category, linked to invoices and orders; no binary storage or OCR
Pre-bid risk score	Implemented	Rule-based score on the compliance screen
Buyer and zone risk profile	Partial	Buyer risk indicators and a delay report built from the vendor's own invoices
Quota-compliance dashboard	Partial	Proxy: share of tracked tenders that are MSME-eligible
Exportable reports	Partial	CSV export and print view; no PDF or Excel generation

SRS feature	Status	Where in the prototype
Authentication and identity	Partial	Internet Identity hook, login and registration screens; no Udyam verification
Role-based access	Partial	Authorisation mixin on the canister; staff roles modelled but not enforced in the UI
WhatsApp alerts, escalation ladder, chatbot	Not implemented	In-app alerts only; escalation steps exist in the data model
Tally, GST, TReDS integrations	Not implemented	Planned for Phase 2
Module 7 collective intelligence	Not implemented	Requires a vendor base
Offline-first mode	Partial	Demo mode persists data in the browser

Table 5. Traceability of SRS features to the prototype.

The prototype therefore covers the Phase 1 core and a thin slice of Phase 2, which is consistent with the SRS's own phasing logic.

---

4. System Architecture and Design

4.1 Reference Architecture in the SRS

The SRS prescribes a **layered modular monolith**: a single Java 17 Spring Boot application whose internal modules correspond to the nine functional modules, backed by PostgreSQL through Spring Data JPA and Hibernate, secured with Spring Security and JWT, scheduled with Spring Scheduler or Quartz, and fronted by a React application [11]. This choice is argued on practical grounds. A modular monolith is faster for a small team to build, test and deploy correctly than a set of microservices, while preserving a path to extract a module, such as Collective Intelligence, later [21]. Table 6 summarises the SRS layers.

Layer	Responsibility	SRS technology
Presentation	Web UI, forms, dashboards, alert display	React.js, Tailwind CSS
API	REST endpoints for web and future mobile clients	Spring Web REST controllers
Business logic	45-day clock, interest, risk scoring, alert triggers	Spring service layer
Security	Authentication, RBAC, consent	Spring Security, JWT
Scheduling	Day-30 and day-40 alerts, nightly aggregation	Spring Scheduler or Quartz
Persistence	Data access and ORM	Spring Data JPA, Hibernate
Database	Users, invoices, documents, alerts, compliance records	PostgreSQL
Integration	Tally, GST e-invoice, WhatsApp Business, TReDS	Java REST and SOAP clients

Table 6. The SRS reference architecture.

4.2 The Implemented Architecture

The prototype that was actually built follows the same *layering* but a different *technology stack*. It is important to state this plainly: the backend is **not** a Java Spring Boot service. It is a Motoko canister (a smart contract in the Internet Computer's terminology) built with the Caffeine toolchain, and the frontend is a React 19 and TypeScript single-page application built with Vite. The frontend contains a configurable REST client with bearer-token handling, which is the seam where a Spring Boot backend would be attached.

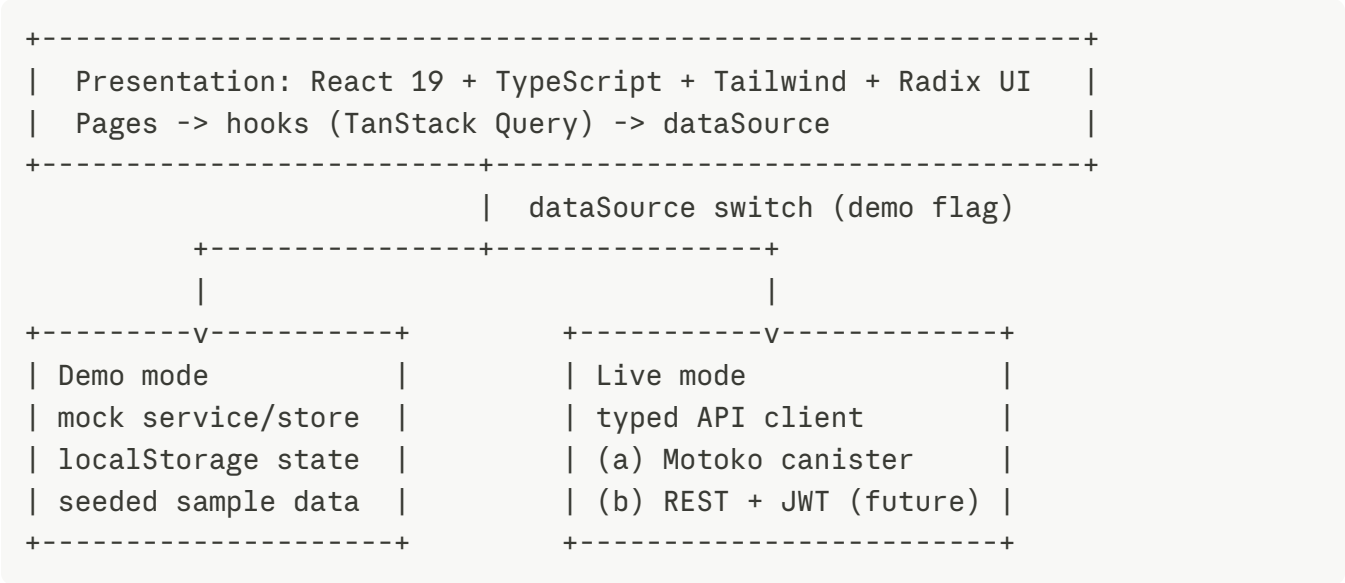


Figure 1. Implemented layering. The `dataSource` module routes every call to either the mock service or the live API depending on a persisted demo-mode flag.

Table 7 maps each SRS layer to its counterpart in the prototype.

SRS layer	SRS technology	Prototype counterpart
Presentation	React, Tailwind	React 19, Tailwind 3, Radix UI primitives, shadcn-style components
API	Spring REST controllers	Canister public methods (mixins) and a typed REST client
Business logic	Spring services	Motoko library modules, plus TypeScript utilities in the client
Security	Spring Security, JWT	Internet Identity authentication hook, authorisation mixin, bearer-token support in the REST client
Persistence	JPA, Hibernate	Persistent actor state in per-user maps with a migration chain
Database	PostgreSQL	Canister memory (live) or browser localStorage (demo)
Scheduling	Spring Scheduler	Not implemented; alerts are derived on screen
Integration	Java clients	Not implemented

Table 7. Mapping from the SRS reference stack to the implemented stack.

Why the difference? The prototype was built to be demonstrable end to end without provisioning a database server, an identity provider or a hosting environment. The cost is a deviation from the SRS's Java mandate; Appendix D therefore proposes a concrete mapping of each canister module to a Spring Boot component so the design can be re-hosted without redesign.

4.3 Repository Structure

The project is a small monorepo. The backend lives under `src/backend` with four kinds of module: `types` (record and variant definitions per domain), `lib` (pure domain logic), `mixins` (the public API surface, one per domain) and `migrations` (state migrations). The frontend lives under `src/frontend/src` and is organised into `pages`, `components`, `hooks`, `services`, `utils`, `layouts`, `routes`, `types` and `__tests__`. This separation keeps domain logic free of transport concerns on both sides of the wire.